Final Project: Particle Filter SLAM

Instructions

Implement simultaneous localization and mapping (SLAM) using encoder and IMU odometry, and 2-D LiDAR scans from a differential-drive robot. Use the odometry, and LiDAR measurements to localize the robot and build a 2-D occupancy grid map of the environment.

- **Robot**: A description of the differential-drive robot is provided in docs/RobotConfiguration.pdf. Fig. 1 shows a schematic of our robot. Even though the description file says that the origin of the robot body frame is at the back axle, it is arbitrary and you can choose your own origin. Using the geometric center of the robot body might yield better results because the differential-drive motion model assumes that the robot is rotating around its center.

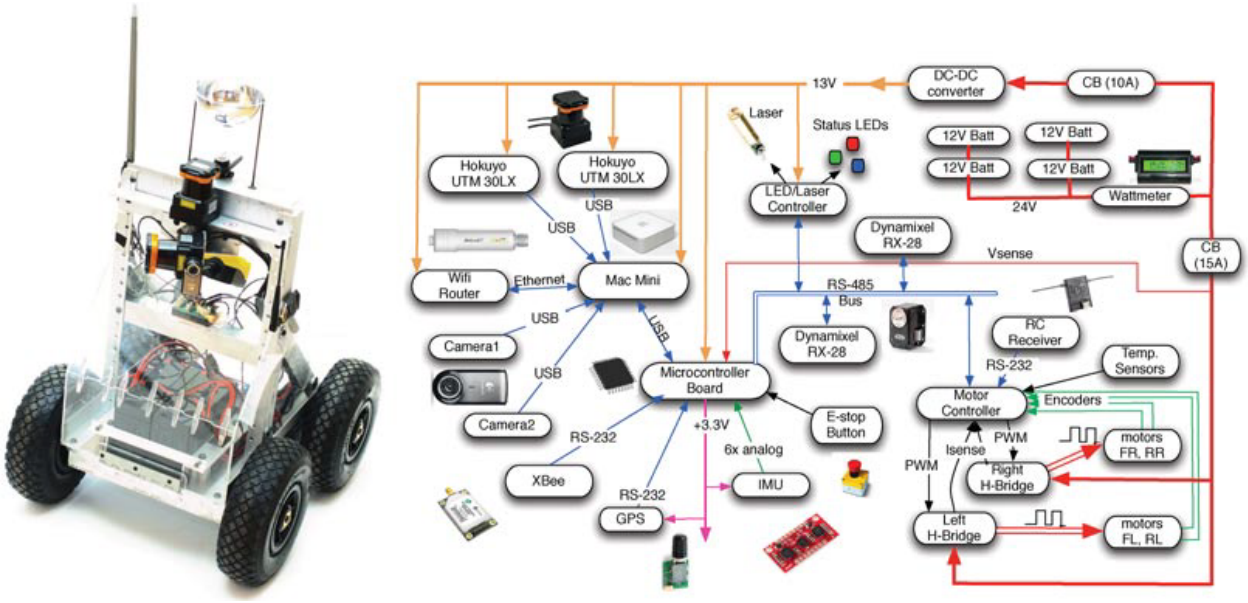

Figure 1: Differential-drive robot, equipped with encoders, IMU, 2-D LIDAR scanner, and an RGBD camera.

- **Sensors**: The robot is equipped with a set of sensors described below. Each sensor reports measurements at its own frequency and, hence, **the sensor data is not synchronized**. The sensor measurements are associated with unix time stamps, which should be used to sort and associate the measurements.
  - **Encoders**: Encoders count the rotations of the four wheels at 40 Hz. The encoder counter is reset after each reading. For example, if one rotation corresponds to ℓ meters traveled, five consecutive encoder counts of $0, 1, 0, -2, 3$ correspond to $(0 + 1 + 0 - 2 + 3)\ell = 2\ell$ meters traveled for that wheel. The data sheet indicates that the wheel diameter is 0.254 m and since there are 360 ticks per revolution, the wheel travels 0.0022 meters per tic. Given encoder counts $[FR,\ FL,\ RR,\ RL]$ corresponding to the front-right, front-left, rear-right, and rear-left wheels, the right wheels travel a distance of $(FR+RR)/2*0.0022$ m, while the left wheels travel a distance of $(FL+RL)/2*0.0022$ m.
  - **IMU**: Linear acceleration and angular velocity data is provided from an inertial measurement unit. The IMU data is noisy, since it is collected from a moving robot with high-frequency vibrations. You should consider applying a low-pass filter (e.g., with bandwidth around 10 Hz) to reduce the measurement noise. We will only use is the **yaw rate** from the IMU as the angular velocity in the differential-drive model in order to predict the robot motion. It is not necessary to use the other IMU measurements.

  - **Hokuyo**: A horizontal LiDAR with 270° degree field of view and maximum range of 30 m provides distances to obstacles in the environment. Each LiDAR scan contains 1081 measured range values. The sensor is called Hokuyo UTM-30LX and its specifications can be viewed online. The location of the sensor with respect to the robot body is shown in the provided robot description file. Make sure you know how to interpret the LiDAR data and how to convert from range measurements to (x, y) coordinates in the sensor frame, then to the body frame of the robot, and finally to the world frame.

- **Particle-filter SLAM**: The first part of the project is to use a particle filter with a differential-drive motion model and scan-grid correlation observation model for simultaneous localization and occupancy-grid mapping. Here is an outline of the necessary operations:

  - **Mapping**: Assume the robot starts with identity pose and use the first LiDAR scan to create an occupancy grid map. Display the occupancy grid map to make sure that your transforms are correct before you start estimating the robot pose. Remove scan points that are too close or too far from the robot. Transform the LiDAR points from the LiDAR frame to the world frame. Use bresenham2D or OpenCV's drawContours[1] to obtain the occupied cells and free cells that correspond to the LiDAR scan. Increase the log-odds of the occupied cells and decrease the log-odds of the free cells in the occupancy-grid map.
  - **Prediction**: Use linear velocity v_t obtained from the encoders and yaw rate ω_t obtained from the IMU to predict the robot motion via the differential-drive motion model. To verify that your prediction step is working correctly, implement dead-reckoning first (prediction with no noise and only a single hypothesis for the robot pose) and plot the robot trajectory. You can also build a complete 2-D occupancy-grid map by combining dead-reckoning and mapping before implementing the particle filter. Once you convince yourself that the mapping and prediction operations work correctly, implement the particle-filter prediction step. Introduce N particles at the initial identity robot pose and add noise to the motion model to obtain a predicted pose $\mu_{t+1|t}^{(i)}$ for each particle $i = 1, \ldots, N$. Apply the motion model repeatedly to obtain N separate particle trajectories and visualize them.
  - **Update**: Once the prediction-only filter works, include an update step that uses scan-grid correlation to correct the robot pose. Remove scan points that are too close or too far. Try the update step with only $3 - 4$ particles at first to see if the weight updates make sense. Transform the LiDAR scan to the world frame using each particle's pose hypothesis. Compute the correlation between the world-frame LiDAR scan and the occupancy map using the mapCorrelation function. Call mapCorrelation with a grid of values (e.g., 9×9) around the current particle position to get a good correlation (see utils.py). You should consider adding variation in the yaw angle of each particle to get good results. Once you obtain the correlation, update the particle weights according to the particle filter equations. If the number of effective particles N_{eff} falls below a threshold, resample the particles.

- **Factor graph in GTSAM:** Begin by constructing a factor graph consisting of nodes representing robot poses at different time steps and edges representing relative pose measurements between these poses obtained from particle filter localization.

- **Loop closure detection:** Use the suggested methods below to introduce loop closure factors into the factor graph. Use GTSAM to optimize the factor graph. This optimization process seeks to minimize the discrepancy between the measured relative poses and estimated absolute poses, thus refining the robot trajectory.

  1. **Fixed-interval loop closure:** After every fixed number of robot poses (e.g., every 10 poses), introduce a loop closure constraint by adding an edge in the factor graph. Use the ICP[2] scan matching algorithm to estimate the relative pose between the two (10-poses-apart) LiDAR scans, ensuring the loop closure is physically plausible.

[1] https://docs.opencv.org/4.7.0/d4/d73/tutorial_py_contours_begin.html
[2] Slide 12: https://natanaso.github.io/ece276a/ref/ECE276A_6_LocalizationOdometry.pdf

2. **Proximity-based loop closure:** Detect potential loop closures by examining poses that are in close proximity. Compare LiDAR scans at these nearby poses – if the scans match well, this indicates a loop closure. Add an edge in the factor graph to represent this loop-closure constraint. Optionally, you may also consider matching between LiDAR scans and the occupancy map obtained from the prior robot trajectory.

- **Applying the optimized trajectory:** Given the optimized trajectory from GTSAM, reconstruct the occupancy grid map and the texture map. The refined poses allow for more accurate mapping, as they correct cumulative errors in the robot's path estimation. Write a project report describing your approach to the SLAM problem.